

Week 3 Progress Summary: Greedy Schemes & Path Simulations

Overview

This week focused heavily on string and array-based optimization problems. The solutions utilized state tracking, boundary checks, index parity analysis, and greedy simulation to manage pathing constraints under specific performance targets. A total of 7 problems were successfully implemented in C++.

Problem Summaries

1. Max Plus Size

- Core Concept: Parity Splitting & Maximum Selection
- Approach: The problem requires picking non-adjacent elements to maximize the sum of the count of picked elements plus their maximum value. Since elements can be chosen optimally by selecting either all even-indexed elements or all odd-indexed elements, the code tracks the maximum element for both sets independently during a linear scan. It computes the score for both alternating patterns based on their respective counts and outputs the maximum score found.

2. Mocha and Red and Blue

- Core Concept: Interval Filling / Forward-Backward Propagation
- Approach: Aims to minimize adjacent identical color pairs in a string with missing entries (?). It finds the first predefined color index as a reference baseline. If none exist, it seeds the first character with 'B'. From that baseline index, it propagates alternating characters forward to the end of the string, and then sweeps backward from the baseline to the start, maintaining a flawless, alternating sequence.

3. Odd Subarrays

- Core Concept: Greedy Inversion Matching
- Approach: Finds the maximum number of disjoint subarrays that contain an inversion (where $p[i] > p[i+1]$). The code iterates through the permutation greedily. Whenever it identifies a local inversion between adjacent elements, it increments the count and forces a jump of two indices forward. This action immediately isolates that inversion and leaves subsequent elements available for separate pairs.

4. Stone Game

- Core Concept: Multi-Option Boundary Partitioning
- Approach: Solves for the minimum number of moves to eliminate both the absolute minimum and maximum elements from an array by peeling from the left or right edges. After tracking the locations of both extremes, it standardizes their positions so that pos1 is the leftmost and pos2 is the rightmost. It then calculates the distance cost for three alternative strategies: removing both from the left, removing both from the right, or splitting the approach from both directions.

5. Test of Love

- Core Concept: Greedy State-Machine / Path Simulation
- Approach: Simulates navigating a river bank containing logs (L), water (W), and crocodiles (C) using jump range m and swim capacity k . The code pads the boundaries and implements a conditional loop. When on a log, it scans ahead up to m steps to prioritize the furthest accessible log. If it must land in water, it aims for the furthest viable water

cell to minimize swimming. It tracks remaining swim stamina k sequentially when stepping through water cells.

6. Thorns and Coins

- Core Concept: Early Termination Tracking
- Approach: Implements a single-pass loop to collect coins (@) along a path. It uses an early termination check to identify an unpassable trap. Because a player can jump over a single thorn (*) but cannot cross two consecutive thorns, the code halts iteration instantly using a lookback condition (`s[i] == '*' && s[i-1] == '*'`).

7. Ugu

- Core Concept: State-Transition Counting
- Approach: Tracks the minimum operations to sort a binary string when an operation flips all subsequent bits. It searches for the initial inversion point where a '1' is followed by a '0'. Once this first inversion is activated, any subsequent character transition (`s[i] != s[i+1]`) signifies a boundary that will need to be inverted back into place, accumulating the operation count for each step change.